

AULA 15

Herança de Classes

1. CRIAÇÃO DE LIVROS IMPRESSOS

A plataforma PlayLivros surgiu com o propósito de transformar a forma como as pessoas consomem livros, disponibilizando audiobooks acessíveis e de alta qualidade para usuários com pouco tempo disponível ou que preferem ouvir conteúdos enquanto realizam outras atividades. Com o passar dos anos, a plataforma consolidou sua presença no mercado e tornou-se uma referência no segmento de livros narrados.



À medida que a plataforma expandia sua base de clientes, foi possível perceber um novo interesse por parte do público: a procura pelas versões físicas das obras disponíveis em áudio. Muitos usuários desejavam colecionar exemplares impressos, presentear amigos e familiares ou reviver o hábito tradicional da leitura em papel, marcando páginas e destacando trechos importantes. Outros ainda demonstravam interesse em combinar as duas experiências, utilizando o audiobook em momentos de praticidade e o livro físico para uma leitura mais aprofundada.



Diante desse cenário, surgiu a proposta de disponibilizar, para cada audiobook da plataforma, a possibilidade de compra da respectiva edição impressa, oferecendo inclusive diferentes opções de capa e encadernação. Com essa iniciativa, surge também um novo desafio: adaptar a estrutura atual do sistema para suportar a venda de livros impressos sem comprometer o funcionamento já consolidado da plataforma de audiobooks.



1.1. ADAPTAÇÃO DA CLASSE LIVRO

Com a decisão de disponibilizar livros impressos na plataforma, a equipe de desenvolvimento inicialmente considerou reutilizar a estrutura já implementada na classe `Livro`, isso porque diversos elementos são compartilhados entre os dois formatos, como título, autor, preço e avaliações. **Mas como distinguir um livro impresso de um audiobook?**



Uma maneira prática de resolver isso é incluir um atributo, por exemplo, um valor booleano que informe se o livro é impresso ou não. Qualquer livro que não possua o atributo de impresso especificado será tratado como um audiobook.

Essa solução é simples e pode até atender às necessidades iniciais da plataforma. No entanto, livros impressos possuem características bastante distintas dos audiobooks, como a necessidade de suportar diferentes opções de capa e encadernação, número de páginas e demais atributos específicos do produto impresso.



Além disso, a entrega de um livro físico envolve custos logísticos independentes do valor da obra. Mesmo que o conteúdo já esteja pronto, ainda há despesas com embalagem, transporte e distribuição para que o produto chegue ao consumidor em segurança. Por isso, cobraremos uma taxa de envio do livro calculada como um percentual do preço (por exemplo, 7%):

```
public double getTaxaEnvio() {  
    return preco * 0.07;  
}
```

Mas quando se trata de um audiobook, a regra é um pouco diferente: eles são digitais e não precisam de envio. Podemos resolver esse problema adicionando uma condição ao método:

```
public double getTaxaEnvio() {  
    if (impresso) {  
        return preco * 0.07;  
    }  
    return 0;  
}
```

Repare que todas essas novas informações pertencem ao livro impresso e não necessariamente ao audiobook. Estamos sobrecarregando a classe `Livro` com atributos e métodos que serão utilizados apenas quando o tipo do livro for um livro impresso.

Podemos deixar a classe `Livro` com essa forma mais genérica, mantendo nela as opções de capa e de encadernação e o número de páginas. Caso o livro não fosse um livro impresso, deixaríamos estes atributos vazios.

Essa é uma possibilidade, porém podemos começar a ter muito atributos opcionais, e a classe ficaria estranha. E em relação aos métodos? A classe `Livro` teria o método `getTaxaEnvio`, que não faz sentido existir em um livro que não é impresso.

1.2. CRIAÇÃO DE LIVROIMPRESSO

Se livro impresso é um elemento importante para o nosso sistema, possuindo atributos e métodos específicos, ele pode e deve ser representado como um objeto! Vamos fazer essa alteração, basta criar a classe `LivroImpresso` e declarar seus atributos:



O código já está melhor organizado, pois a classe `Livro` não precisa mais guardar atributos e métodos que só fazem sentido para livros impressos. Com isso, cada classe passa a ter responsabilidades mais específicas, tornando a estrutura do sistema mais clara.

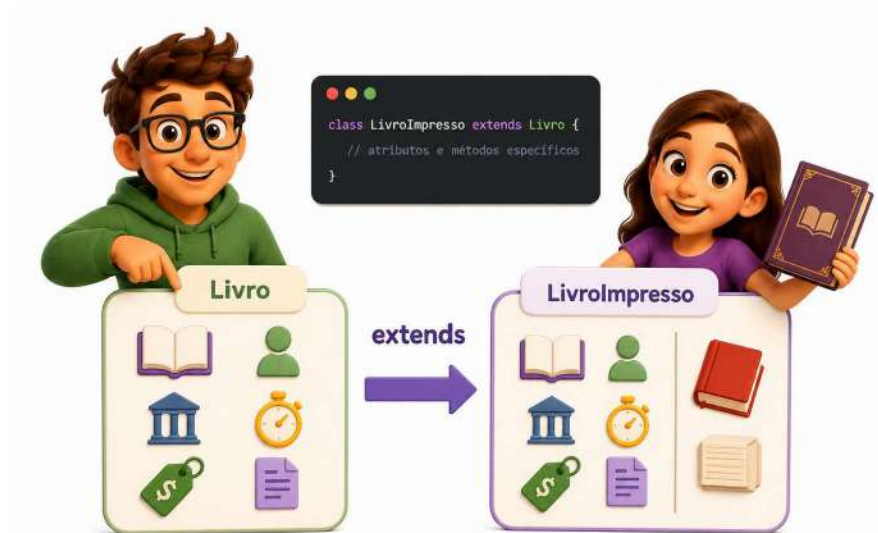
No entanto, ainda existe um ponto que pode ser aprimorado. Para representar os comportamentos de um livro impresso, acabamos repetindo vários atributos e métodos que já haviam sido definidos na classe `Livro`. Essa repetição aumenta a quantidade de código e pode tornar futuras alterações mais trabalhosas, já que mudanças em funcionalidades comuns precisariam ser feitas em mais de um lugar.

Se tivéssemos um outro tipo de livro que tem características diferentes do áudiobook, precisaríamos criar uma outra classe e copiar o código novamente! Além disso, se um dia precisarmos adicionar uma nova informação para todos os livros, precisaremos passar por todas as classes de livro e adicionar esse atributo. O problema acontece novamente por não centralizarmos as informações principais do livro em um único lugar!

1.3. HERANÇA DE LIVRO

Existe um jeito, em Java, de relacionarmos uma classe de tal maneira que uma delas herda tudo que a outra tem, indicando uma relação de classe mãe e classe filha. No nosso caso, gostaríamos de fazer com

que o `LivroImpresso` tivesse tudo que um `Livro` tem, gostaríamos que ela fosse uma extensão de `Livro`. Fazemos isto através da palavra chave `extends`.



Ao utilizar a palavra reservada `extends`, estamos dizendo que um `LivroImpresso` (subclasse) herda tudo o que a classe `Livro` (superclasse) tem. Por meio dela, uma classe pode aproveitar atributos e métodos já existentes em outra classe, promovendo reutilização de código, organização e manutenção mais eficiente do software.

Essa organização resolve os problemas anteriores: **evita a mistura de responsabilidades e elimina a duplicação de atributos**. Além disso, permite que a plataforma continue crescendo, podendo incluir, no futuro, outros formatos de livros de forma estruturada e clara.

1.3.1. EXERCÍCIOS: HERANÇA

- 1) Crie, no pacote `model`, a classe `LivroImpresso` que herda de `Livro`.

```
public class LivroImpresso extends Livro {  
  
}
```

- 2) Defina o atributo para o número de páginas.

```
public class LivroImpresso extends Livro {  
    private int paginas;  
}
```


- 3) Localize a classe `TestaLivros` e inclua, dentro do `main`, um livro impresso. Observe que o objeto do tipo `LivroImpresso` tem os *setters* para os atributos declarados em `Livro`, mesmo sem ter nenhum desses métodos declarados diretamente em `LivroImpresso`.

```
public class TestaLivros {
    static void main() {

        ...

        LivroImpresso quartoLivro = new LivroImpresso();
        quartoLivro.setTitulo("Pai Rico, Pai Pobre");
        quartoLivro.setAutor("Robert Kiyosaki");
        quartoLivro.setNarrador("Marcelo Sanches");
        quartoLivro.setEditora("Alta Books");
        quartoLivro.setDuracao(LocalTime.of(6, 15));
        quartoLivro.setVersao(VersaoEnum.INTEGRAL);
        quartoLivro.setPreco(19.90);
        quartoLivro.setPaginas(197);
        quartoLivro.setResumo("Ensina conceitos fundamentais sobre educação financeira.");

    }
}
```

- 4) Execute a classe para ver a saída.
- 5) Execute os comandos para envio das alterações ao *github*:

```
git status
git add .
git commit -m "Criação de classe LivroImpresso - herança de Livro"
git push
```

1.4. EVOLUÇÃO DA MODELAGEM: CLASSE AUDIOBOOK

Na estrutura atual, deixamos os comportamentos específicos de um `LivroImpresso` bem encapsulados em sua classe, mas repare que há um problema em nossa herança: a classe `Livro` concentra atributos e comportamentos que são características exclusivas de audiolivros. Como consequência, a classe `LivroImpresso`, ao herdar de `Livro`, passa a carregar informações que não possuem significado para sua natureza, como **narrador** e **duração**.

Essa situação caracteriza uma generalização inadequada, pois a superclasse `Livro` contém elementos que não seria comuns a todas as suas subclasses. Um livro impresso não possui narrador nem tempo de reprodução, da mesma forma que um audiolivro não possui número de páginas físicas ou custos de envio. Dessa forma, a herança acaba introduzindo atributos irrelevantes em determinadas especializações, tornando o modelo menos coeso e potencialmente gerando valores nulos, validações desnecessárias e maior complexidade de manutenção.

Mais uma vez entramos na regra: se um elemento é importante e tem regras específicas, ele deve ser representado como um objeto. Podemos criar uma nova classe em nosso projeto, para representar tudo o que um audiobook tem e como ele se comporta, adequando a modelagem ao princípio da responsabilidade única e à correta representação do domínio.



Ao introduzir a classe `AudioBook`, os atributos `narrador`, `duracao` e `versao` podem ser movidos para essa nova especialização, mantendo na classe `Livro` apenas as características compartilhadas por todos os tipos de livro, como título, autor, editora, preço, resumo, classificação indicativa, data de lançamento e avaliações. Com isso, cada subclasse passa a conter apenas os dados e comportamentos pertinentes ao seu contexto, sem sobrecarregar a classe base com atributos específicos de determinados formatos de publicação.

1.4.1. EXERCÍCIOS: AUDIOBOOK

- 1) Crie, no pacote `model`, a classe `AudioBook` que herda de `Livro`.

```
public class AudioBook extends Livro {  
    }  
}
```

- 2) Defina os atributos `narrador`, `duracao` e `versao`.

```
public class AudioBook extends Livro {  
    private String narrador;  
    private LocalDateTime duracao;  
    private String versao;  
}
```

```
private VersaoEnum versao;
```

```
}
```

- 3) Gere os *setters* para os atributos.
- 4) Remova, agora, os atributos `narrador`, `duracao` e `versao` da classe `Livro`, pois eles são características de um `AudioBook` apenas. Remova também os *setters* e a exibição do campo `narrador` no método `exibir()`.

- 5) Abra a classe `TestaLivros` e defina no lugar de `Livro` um `AudioBook`.

```
Livro primeiroLivro = new Livro();  
AudioBook primeiroLivro = new AudioBook();
```

- 6) Execute a classe para ver a saída.
- 7) Execute os comandos para envio das alterações ao *github*:

```
git status
```

```
git add .
```

```
git commit -m "Criação de classe AudioBook - herança de Livro"
```

```
git push
```

1.5. MODIFICADOR PROTECTED

Vimos que na entrega de um livro físico cobraremos uma taxa de envio calculada com um percentual de 7% sobre o preço do livro:

```
public double getTaxaEnvio() {  
    return preco * 0.07;  
}
```

O método acima não compila, pois a visibilidade do atributo `preco` da classe `Livro` é `private`. Afinal, um atributo `private` só pode ser acessado pela própria classe, nem mesmo as classes filhas (subclasses) podem violar essa regra.

Mas para acessar o atributo `preco` que herdamos, como faríamos? Não gostaríamos de deixar o atributo de `Livro` como `public`, pois dessa maneira qualquer um poderia alterar o atributo dos objetos deste tipo.

Uma alternativa é modificar a visibilidade do atributo para `protected`, que fica entre o `private` e o `public` e só pode ser acessado (visível) pela própria classe e por suas subclasses, além de classes dentro do próprio pacote.

1.5.1. EXERCÍCIOS: PROTECTED

- 1) Abra a classe `Livro` e defina o atributo `preço` com o modificador `protected`.

```
public class Livro {  
    protected double preço;  
}
```

- 2) Abra a classe `LivroImpresso` e defina o método `getTaxaEnvio()`.

```
public double getTaxaEnvio() {  
    return preço * 0.07;  
}
```

- 3) Execute os comandos para envio das alterações ao *github*:

```
git status  
git add .  
git commit -m "Criação de metodo getTaxaEnvio() em LivroImpresso"  
git push
```

1.6. SOBRESCRITA DE MÉTODOS

Como requisito da implementação, a classe `Livro` deverá disponibilizar o método `getTotal()`, que encapsula a lógica de cálculo do total e retorna o resultado ao usuário da classe.

```
public double getTotal() {  
    return preço;  
}
```

Suponha que a classe `LivroImpresso` precise considerar a taxa de envio em seu valor total. Se deixarmos a classe como ela está, ela vai herdar o método `getTotal`.

```
LivroImpresso livro = new LivroImpresso();  
livro.setPreco(100.00);  
System.out.println(livro.getTotal());
```

O resultado aqui será 100. Não queremos essa resposta, pois o `LivroImpresso` deveria custar 107 reais com taxa de entrega nesse caso.

Para consertar isso, uma das opções seria criar um novo método na classe `LivroImpresso` chamado `getTotalLivroImpresso()`, por exemplo. O problema é que teríamos dois métodos em `LivroImpresso`, confundindo bastante quem for usar essa classe, além de que cada um dá uma resposta diferente.

No Java, quando herdamos um método, podemos alterar seu comportamento reescrevendo (reescrever, sobrescrever, *override*) este método. Embora a implementação padrão de `getTotal()`

retorne apenas o preço do livro, classes derivadas podem sobrescrever esse método para incluir regras específicas de negócio, como acréscimos, descontos ou custos adicionais.

```
@Override  
public double getTotal() {  
    return preco + getTaxaEnvio();  
}
```

Em Java, a anotação `@Override` é utilizada para indicar explicitamente que um método está sobrescrevendo um método herdado, auxiliando na legibilidade e na detecção de erros pelo compilador.

Uma anotação é apenas uma marcação, como um post-it com uma observação importante sobre o local anotado, existindo diversas anotações e cada uma vai ter um efeito diferente sobre o código.

A sobrescrita permite que uma classe filha altere ou especialize o comportamento herdado de uma classe pai. Assim, embora o método mantenha o mesmo nome e a mesma assinatura, sua implementação pode ser diferente para atender às necessidades específicas da subclasse.

Por mais que um `LivroImpresso` seja um `Livro`, quando alguém chamar o método `getTotal()`, o valor será de 107 reais e não de 100 reais com o que está definido na classe mãe. A JVM sempre vai procurar o método primeiro no objeto que foi instanciado e, apenas quando não encontrar, procurará em sua superclasse.



1.6.1. EXERCÍCIOS: SOBRESCRITA DE MÉTODO



- 1) Abra a classe `Livro` e defina o método `getTotal()` a seguir:

```
public double getTotal() {  
    return preco;  
}
```

- 2) Abra a classe `LivroImpresso`, clique com o botão direito no final da classe e acesse **Generate → Override Methods...** Escolha a opção `getTotal()` e finalize com **OK**.

- 3) Codifique o método considerando a soma do preço com a taxa de envio calculada.

```
@Override  
public double getTotal() {  
    return preco + getTaxaEnvio();  
}
```

- 4) Execute os comandos para envio das alterações ao *github*:

```
git status  
git add .  
git commit -m "Criação de metodo getTotal()"  
git push
```